

Universidad de las Fuerzas Armadas – ESPE

Departamento de Ciencias de la Computación

Ingeniería en Tecnologías de la Información

Modelado UML — Diagrama de casos de uso, Diagrama Puente

Grupo 1

Integrantes

Eatan Joao Ayala Maldonado

Johan Mateo Catota Analuisa

Lenin Esteban Cuichan Males

Katerine Julisa Caizaluisa Guzman

Metodología de Desarrollo Software

30693

Ing. Jenny Ruiz

13 de Mayo 2026

Contenido

1. Listado de Requisitos Funcionales “Huertos el JuanCa”	3
1.2 Tabla de Requisitos Funcionales por Sprint	4
2. Sprint 1	5
2.1 Diagrama de Casos de Uso REQ001 – REQ005	5
• <i>Código PlantUML</i>	5
• <i>Captura del Diagrama PlantUML Sprint 1</i>	6
2.2 Puente Sprint 1	6
3.2.2 Diagrama Puente PlantUML Sprint 1	7
3. Sprint 2	8
3.1 Diagrama de Casos de Uso REQ006 -REQ009	8
• <i>Código PlantUML</i>	8
• <i>Captura de Diagrama PlantUML Sprint 2</i>	9
3.2 Puente Sprint 2	10
3.2.2 Diagrama Puente PlantUML Sprint 2	10
4. Sprint 3	11
4.1 Diagrama de Casos de Uso REQ010 – REQ011	11
• <i>Código UML</i>	11
• <i>Captura del Diagrama de Casos de Uso Sprint 3</i>	13
4.2 Puente Sprint 3	13
4.2.2 Diagrama Puente PlantUML Sprint 3	13
5. Sprint 4	14
5.1 Diagrama de Casos de Uso REQ012- REQ013	14
• <i>Código PlantUML</i>	14
• <i>Captura de Diagrama de Casos de Uso Sprint 4</i>	16
5.1 Puente Sprint 4	16
5.2.2 Diagrama Puente PlantUML Sprint 4	16

1. Listado de Requisitos Funcionales “Huertos el JuanCa”

Tabla 1 Requisitos Funcionales

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
[REQ001]	El sistema debe permitir navegar entre los diferentes módulos del aplicativo.	Administrador del Sistema	Navegar por el Sistema
[REQ002]	El sistema debe permitir registrar productos con información de nombre, precio, stock y disponibilidad.	Administrador del Sistema	Registrar Producto
[REQ003]	El sistema debe permitir modificar o eliminar productos registrados mediante un catálogo.	Administrador del Sistema	Gestionar Producto
[REQ004]	El sistema debe permitir registrar clientes con sus datos personales y dirección.	Administrador del Sistema	Registrar Cliente
[REQ005]	El sistema debe permitir actualizar o eliminar información de clientes registrados.	Administrador del Sistema	Gestionar Cliente
[REQ006]	El sistema debe permitir visualizar alertas de cumpleaños de clientes próximos a cumplir años.	Administrador del Sistema	Consultar Alerta de Cumpleaños
[REQ007]	El sistema debe permitir registrar clientes potenciales.	Administrador del Sistema	Registrar Cliente Potencial
[REQ008]	El sistema debe permitir modificar o eliminar clientes potenciales registrados.	Administrador del Sistema	Gestionar Cliente Potencial
[REQ009]	El sistema debe permitir visualizar, reactivar o eliminar clientes inactivos.	Administrador del Sistema	Gestionar Cliente Inactivo
[REQ010]	El sistema debe permitir registrar ventas asociadas a clientes y productos.	Administrador del Sistema	Registrar Venta

[REQ011]	El sistema debe permitir almacenar la información del sistema en archivos Excel.	Administrador del Sistema	Almacenar datos en Excel
[REQ012]	El sistema debe permitir generar e imprimir notas de venta.	Administrador del Sistema	Generar nota de Venta
[REQ013]	El sistema debe permitir generar reportes en formato Excel.	Administrador del Sistema	Generar Reportes Excel

1.2 Tabla de Requisitos Funcionales por Sprint

Tabla 2 Requisitos Funcionales por Sprint

Sprint	Requisitos Funcionales
Sprint 1	• [REQ001] • [REQ002] • [REQ003] • [REQ004] • [REQ005]
Sprint 2	• [REQ006] • [REQ007] • [REQ008] • [REQ009]
Sprint 3	• [REQ010] • [REQ011]
Sprint 4	• [REQ012] • [REQ013]

2. Sprint 1

2.1 Diagrama de Casos de Uso REQ001 – REQ005

- **Código PlantUML**

@startuml Sprint1

left to right direction

skinparam backgroundColor white
skinparam shadowing true
skinparam roundcorner 20

skinparam actorStyle awesome

skinparam actor {
 BackgroundColor #ed4519
 BorderColor #BF9000
 FontColor #000000
 FontStyle bold
}

skinparam rectangle {
 BackgroundColor #ffffff
 BorderColor #000000
 FontColor #000000
 FontStyle bold
 RoundCorner 25
}

skinparam usecase {
 BackgroundColor #8fe4ed
 BorderColor #000000
 FontColor #000000
 FontStyle bold
}

skinparam ArrowColor #000000
skinparam ArrowThickness 2

actor "Administrador\ndel Sistema" as Admin

rectangle "Sprint 1 - Navegación y Funciones Cliente/Producto" {

 usecase "[REQ001] Navegar por el Sistema" as UC1
 usecase "[REQ002] Registrar Producto" as UC2
 usecase "[REQ003] Gestionar Producto" as UC3
 usecase "[REQ004] Registrar Cliente" as UC4
 usecase "[REQ005] Gestionar Cliente" as UC5

}

Admin --> UC1
Admin --> UC2
Admin --> UC3
Admin --> UC4
Admin --> UC5

@enduml

- Captura del Diagrama PlantUML Sprint 1

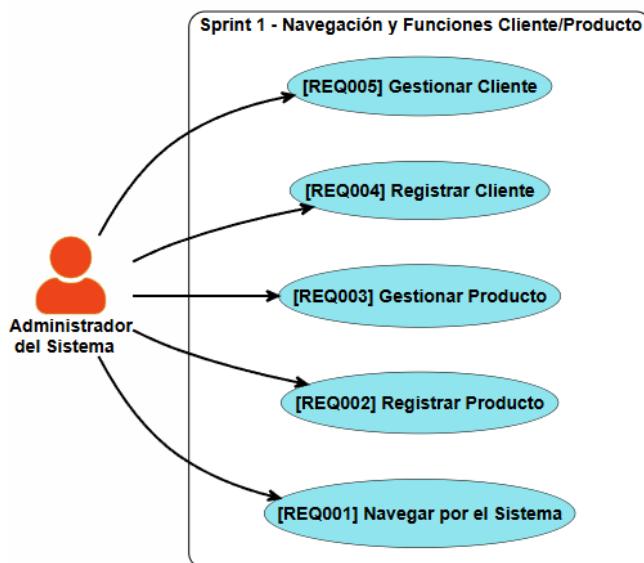


Ilustración 1 Diagrama 1 PlantUML

2.2 Puente Sprint 1

Tabla 3 Puente hacia Clases Sprint 1

Tipo de clase	Propósito en análisis	Ejemplo para CRUD Estudiante
Frontera / Boundary	Representa la interacción entre el usuario y el sistema	Frm_Principal
Control / Control	Coordina la lógica de los casos de uso	ProductoService, ClienteService
Entidad / Entity	Representa la información almacenada del sistema	Producto, Cliente

3.2.2 Diagrama Puente PlantUML Sprint 1

- **Código PlantUML**

@startuml PuenteSprint1

left to right direction

skinparam backgroundColor white

skinparam shadowing false

actor "Administrador\n del Sistema" as Admin

boundary "Frm_Principal" as UI

control "ProductoService" as ProdSrv

control "ClienteService" as CliSrv

entity "Producto" as Producto

entity "Cliente" as Cliente

Admin --> UI : navegar()

UI --> ProdSrv :

guardarProducto()\nmodificarProducto()\neliminarProducto()\nbuscarProducto()

UI --> CliSrv : guardarCliente()\nmodificarCliente()\neliminarCliente()\nbuscarCliente()

ProdSrv --> Producto : gestiona datos

CliSrv --> Cliente : gestiona datos

ProdSrv --> UI : resultados

CliSrv --> UI : resultados

@enduml

- **Captura Diagrama Puente PlantUML Sprint 1**

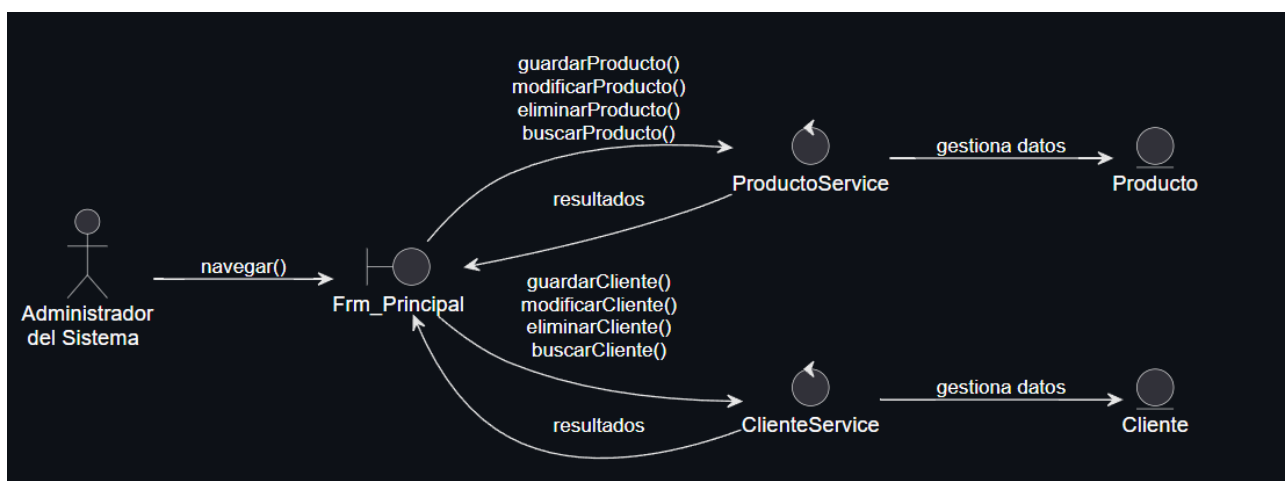


Ilustración 5 Puente PlantUML Sprint 1

3. Sprint 2

3.1 Diagrama de Casos de Uso REQ006 -REQ009

- **Código PlantUML**

@startuml Sprint2

left to right direction

skinparam backgroundColor white

skinparam shadowing true

skinparam roundcorner 20

skinparam actorStyle awesome

skinparam actor {

 BackgroundColor #ed4519

 BorderColor #BF9000

 FontColor #000000

 FontStyle bold

}

skinparam rectangle {

 BackgroundColor #ffffff

 BorderColor #000000

 FontColor #000000

 FontStyle bold

 RoundCorner 25

}

skinparam usecase {

 BackgroundColor #78dbc0

 BorderColor #000000

 FontColor #000000

 FontStyle bold

}

skinparam ArrowColor #000000

skinparam ArrowThickness 2

actor "Administrador\n del Sistema" as Admin

rectangle "Sprint 2 - Gestión y Seguimiento de Clientes" {

usecase "[REQ006] Alerta de Cumpleaños" as UC6

usecase "[REQ007] Registrar Cliente Potencial" as UC7

usecase "[REQ008] Gestionar Cliente Potencial" as UC8

usecase "[REQ009] Gestionar Cliente Inactivo" as UC9

}

Admin --> UC6

Admin --> UC7

Admin --> UC8

Admin --> UC9

@enduml

- **Captura de Diagrama PlantUML Sprint 2**

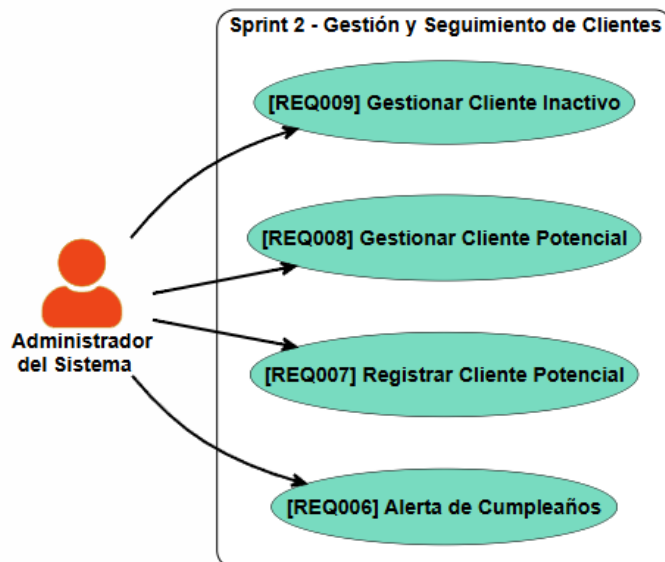


Ilustración 2 Diagrama 2 PlantUML

3.2 Puente Sprint 2

Tabla 4 Puente hacia Clases Sprint 2

Tipo de clase	Propósito en análisis	Ejemplo para CRUD Estudiante
Frontera / Boundary	Representa la interacción entre el usuario y el sistema	Frm_Principal, Frm_CliPotenciales
Control / Control	Coordina la lógica de los casos de uso	ClienteService, ClientePotencialService
Entidad / Entity	Representa la información almacenada del sistema	Cliente, ClientePotenciales

3.2.2 Diagrama Puente PlantUML Sprint 2

- **Codigo PlantUML**

@startuml PuenteSprint2

left to right direction

skinparam backgroundColor white

skinparam shadowing false

actor "Administrador\n del Sistema" as Admin

boundary "Frm_Principal" as MainUI

boundary "Frm_CliPotenciales" as PotUI

control "ClienteService" as CliSrv

control "ClientePotencialService" as PotSrv

entity "Cliente" as Cliente

entity "ClientePotenciales" as Potencial

Admin --> MainUI : cumpleaños()\ninactivos()

Admin --> PotUI : potenciales()

MainUI --> CliSrv : verificarCumpleañosHoy()

PotUI --> PotSrv : guardar()\nmodificar()\neliminar()

CliSrv --> Cliente

PotSrv --> Potencial

@enduml

- Captura Diagrama Puente PlantUML Sprint 2

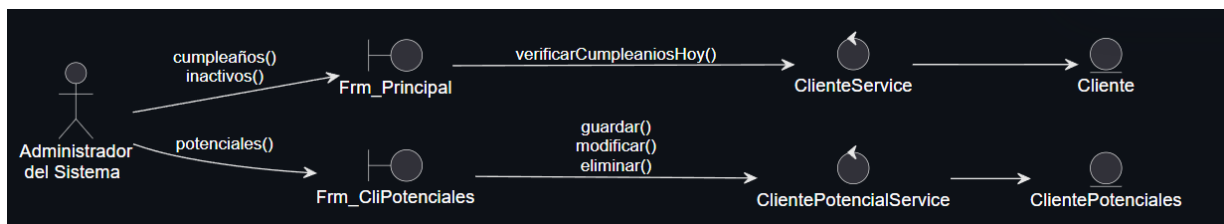


Ilustración 6 Puente PlantUML Sprint 2

4. Sprint 3

4.1 Diagrama de Casos de Uso REQ010 – REQ011

- **Código UML**

@startuml Sprint3

left to right direction

skinparam backgroundColor white

skinparam shadowing true

skinparam roundcorner 20

skinparam actorStyle awesome

skinparam actor {

BackgroundColor #ed4519

BorderColor #BF9000

```
FontColor #000000

FontStyle bold
}

skinparam rectangle {
    BackgroundColor #ffffff
    BorderColor #000000
    FontColor #000000
    FontStyle bold
    RoundCorner 25
}

skinparam usecase {
    BackgroundColor #788bdb
    BorderColor #000000
    FontColor #000000
    FontStyle bold
}

skinparam ArrowColor #000000
skinparam ArrowThickness 2

actor "Administrador\n del Sistema" as Admin

rectangle "Sprint 3 - Ventas y Almacenamiento" {

    usecase "[REQ010] Registrar Venta" as UC10
    usecase "[REQ011] Almacenamiento en Excel" as UC11

}

Admin --> UC10
Admin --> UC11

@enduml
```

- **Captura del Diagrama de Casos de Uso Sprint 3**

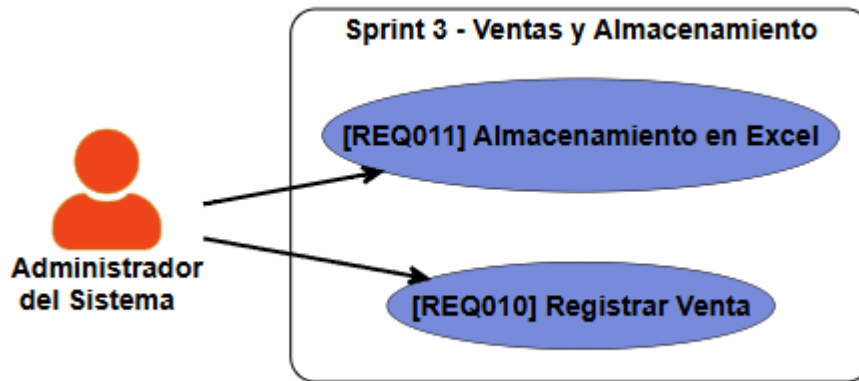


Ilustración 3 Diagrama 3 PlantUML

4.2 Puente Sprint 3

Tabla 5 Puente hacia Clases Sprint 3

Tipo de clase	Propósito en análisis	Ejemplo para CRUD Estudiante
Frontera / Boundary	Representa la interacción entre el usuario y el sistema	Frm_Principal
Control / Control	Coordina la lógica de los casos de uso	VentaService
Entidad / Entity	Representa la información almacenada del sistema	Venta

4.2.2 Diagrama Puente PlantUML Sprint 3

- **Codigo PlantUML**

@startuml PuenteSprint3

left to right direction

actor "Administrador\n del Sistema" as Admin

boundary "Frm_Principal" as UI

control "VentaService" as VentaSrv

entity "Venta" as Venta

Admin --> UI : registrar venta

UI --> VentaSrv : guardar venta

VentaSrv --> Venta : gestiona datos

VentaSrv --> UI : resultados

@enduml

- **Captura Diagrama Puente PlantUML Sprint 3**

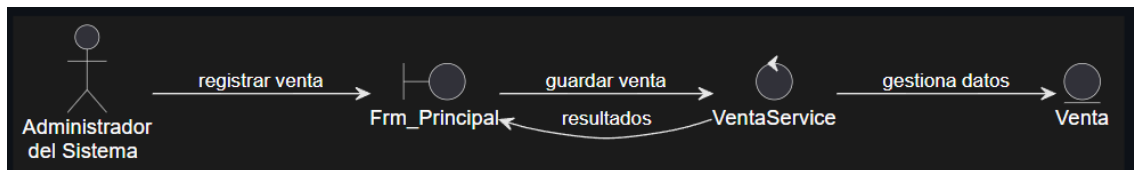


Ilustración 7 Puente PlantUML Sprint 3

5. Sprint 4

5.1 Diagrama de Casos de Uso REQ012- REQ013

- **Código PlantUML**

@startuml Sprint4

left to right direction

skinparam backgroundColor white

skinparam shadowing true

skinparam roundcorner 20

skinparam actorStyle awesome

skinparam actor {

BackgroundColor #ed4519

```
BorderColor #BF9000

FontColor #000000

FontStyle bold
}

skinparam rectangle {
    BackgroundColor #ffffff
    BorderColor #000000
    FontColor #000000
    FontStyle bold
    RoundCorner 25
}

skinparam usecase {
    BackgroundColor #faee48
    BorderColor #000000
    FontColor #000000
    FontStyle bold
}

skinparam ArrowColor #000000
skinparam ArrowThickness 2

actor "Administrador\n del Sistema" as Admin

rectangle "Sprint 4 - Documentos y Reportes" {

    usecase "[REQ012] Generar Nota de Venta" as UC12
    usecase "[REQ013] Generar Reportes Excel" as UC13

}

Admin --> UC12
Admin --> UC13
@enduml
```

- **Captura de Diagrama de Casos de Uso Sprint 4**

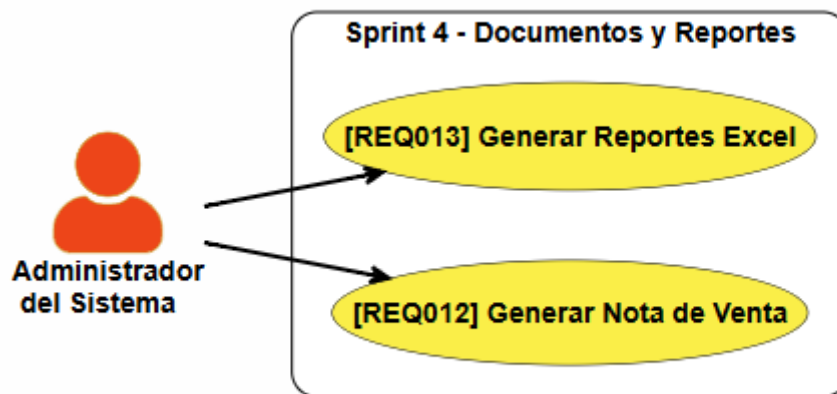


Ilustración 4 Diagrama 4 PlantUML

5.2 Puente Sprint 4

Tabla 6 Puente hacia Clases Sprint 4

Tipo de clase	Propósito en análisis	Ejemplo para CRUD Estudiante
Frontera / Boundary	Representa la interacción entre el usuario y el sistema	Frm_Principal
Control / Control	Coordina la lógica de los casos de uso	ReporteService
Entidad / Entity	Representa la información almacenada del sistema	NotaDeVenta, ReporteExcel

5.2.2 Diagrama Puente PlantUML Sprint 4

- **Código PlantUML**

@startuml PuenteSprint4

left to right direction

actor "Administrador\n del Sistema" as Admin

boundary "Frm_Principal" as UI

control "ReporteService" as ReporteSrv

entity "Reporte" as Reporte

entity "NotaVenta" as Nota

Admin --> UI : generar reportes\nemitir notas

UI --> ReporteSrv : exportar reportes\ngenerar nota

ReporteSrv --> Reporte : genera reportes

ReporteSrv --> Nota : genera documento

ReporteSrv --> UI : resultados

@enduml

- **Captura Diagrama Puente PlantUML Sprint 4**

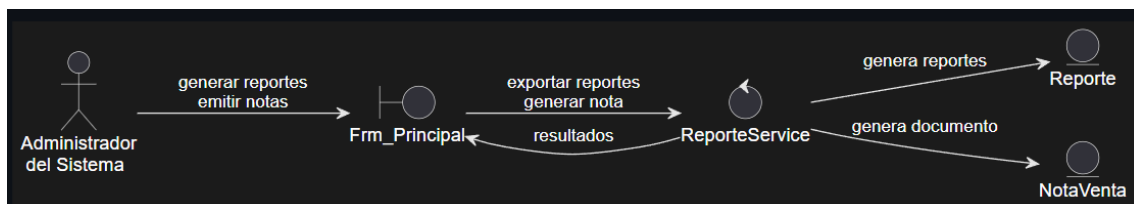


Ilustración 8 Puente PlantUML Sprint 4